

```

// RECURSIVE CASE
let results = [];
if (openRem > 0) { ❹
    document.write(".".repeat(indent) + "Adding open parenthesis.<br />");
    Array.prototype.push.apply(results, getBalancedParens(
        pairs, openRem - 1, closeRem, current + '(', indent + 1));
}
if (closeRem > openRem) { ❺
    document.write(".".repeat(indent) + "Adding close parenthesis.<br />");
    results = results.concat(getBalancedParens(
        pairs, openRem, closeRem - 1, current + ')', indent + 1));
}

// BASE CASE
document.write(".".repeat(indent) + "2nd base case. Returning " + results + "
<br />");
return results; ❻
}

document.write(<pre>"All combinations of 2 balanced parentheses:<br />");
document.write("Results: ", getBalancedParens(2), "</pre>");
</script>

```

The output of these programs looks like this:

```

All combinations of 2 balanced parentheses:
Start of pairs=2, openRem=2, closeRem=2, current=""
Adding open parenthesis.
.Start of pairs=2, openRem=1, closeRem=2, current="("
..Adding open parenthesis.
...Start of pairs=2, openRem=0, closeRem=2, current="(("
...Adding close parenthesis.
....Start of pairs=2, openRem=0, closeRem=1, current="(()"
....Adding close parenthesis.
.....Start of pairs=2, openRem=0, closeRem=0, current="(())"
.....1st base case. Returning ['(())']
....2nd base case. Returning ['(())']
...2nd base case. Returning ['(())']
..Adding close parenthesis.
..Start of pairs=2, openRem=1, closeRem=1, current="()"
..Adding open parenthesis.
...Start of pairs=2, openRem=0, closeRem=1, current="()("
...Adding close parenthesis.
....Start of pairs=2, openRem=0, closeRem=0, current="()()"

```